Chapter 1

Alternative Data and Deep Learning in Finance

1.1 Introduction

Chapter 1 introduced a table of the major categories of financial data this book draws on, and set aside one row, alternative data, non-traditional sources such as satellite imagery, web traffic, or shipping data that proxy for economic activity, for this chapter. Chapter 3 noted that large asset managers already parse filings and calls faster than human analysts; this chapter turns to data sources that did not exist, in usable form, a generation ago, and to the deep learning architectures built to extract signal from them. Chapter 4 observed that alternative data can supplement the delayed, discretionary numbers in financial statements; Chapter 6 introduced the feedforward neural network and its forward pass; Chapter 16 developed the transformer architecture for text. This chapter extends the neural network toolkit to two more data shapes, grid-structured images and ordered sequences, and works through the full pipeline from raw alternative data to a usable financial signal, while treating the biases specific to this kind of data as a first-class topic rather than an afterthought.

Thirteen worked examples run through the chapter: a subscription cost-benefit calculation weighing a data vendor's licensing cost against the expected trading edge it buys; a simple regression of a retailer's revenue growth on a satellite-derived parking-lot car-count proxy; a multi-source regression that combines that satellite proxy with web-traffic and card-transaction growth into an earnings-surprise signal, evaluated the way Chapter 13 evaluates any trading strategy; a direct comparison of that signal built the naive way versus the way that respects real-world data-delivery lag; a peer-relative normalization of the same satellite proxy against a sector benchmark. A hand-computed two-dimensional convolution over a stylized satellite image patch; a hand-computed recurrent neural network forward pass over a short sequence of web-traffic readings; a hand-computed autoencoder reconstruction used to flag a data-quality anomaly; a Loughran-McDonald tone score built directly from a company's own risk-factor disclosures, illustrating both the appeal and the limits of building an ESG-style signal in-house rather than buying one; and, going a level deeper still, a transfer-learning fine-tuning step, a graph neural network update over a shell-company network, a cross-attention fusion of three data modalities into one representation, and a contrastive-pretraining probability calculation showing how a labeled-data-free encoder learns to tell same-image crops apart from different-image crops.

This chapter's structure mirrors the arc of a real alternative-data project, and is worth previewing before working through the details. It begins where any such project begins, deciding what alternative data even is and how a fund acquires and evaluates it (Sections 1.2.1 and 1.2.2), before turning to the analytics itself: a single-source regression, a multi-source signal, the point-in-time discipline that keeps that signal honest, and the seasonal and peer-relative adjustments that make raw readings usable in the first place (Sections 1.2.3 through 1.3.2).

It then turns to the deep learning architectures that extract features from raw images and sequences rather than from a vendor's already-processed number (Section 1.3.3), the data-quality and governance problems

specific to this kind of data (Section 1.3.4), a worked example on building an ESG-style text signal in-house as a partial, auditable response to those same governance problems (Section 1.4.1), and a set of more advanced architectures, transfer learning, graph neural networks, cross-attention fusion, and contrastive pretraining, that represent where the field is moving beyond the basics (Section 1.4.2). A real-world case vignette and a closing set of challenges tie the chapter's techniques back to the genuine economic and regulatory stakes of using data that, in most cases, did not exist in usable form before this decade.

1.2 The Alternative-Data Landscape

This part establishes what alternative data is, who sells it, and works two signals end to end.

1.2.1 What Counts as Alternative Data

Suppose a hedge fund wants to know how a retailer's holiday-quarter sales are trending three months before the retailer itself reports earnings, information that would be extremely valuable if the fund could act on it early and worthless the moment everyone else has it too. Traditional market, fundamental, and macroeconomic data (Chapter 1's Table 1.6) cannot answer this: market data only reflects what the market already knows, and fundamental data about this quarter will not exist until the retailer files it.

Alternative data exists to fill this gap. In financial industry usage, it means any data source used to inform an investment or credit decision that falls outside the traditional trio of market data, fundamental data, and macroeconomic data. The term is a moving target: yesterday's alternative data becomes today's mainstream input once enough market participants adopt it, so the boundary is defined by current practice rather than any fixed technical property. Table 1.1 catalogues the categories most widely used in practice.

Category	Examples	Typical financial use
Satellite and aerial imagery	Parking-lot car counts, crop health indices, oil-storage tank levels, construction progress	Retail sales nowcasting, commodity supply estimation, real-estate tracking
Web and app data	Website traffic, app downloads and reviews, job postings, e-commerce pricing	Revenue nowcasting, hiring-trend and competitive signals
Transaction and card-panel data	Aggregated, anonymized consumer card-spending panels	Same-store sales nowcasting ahead of earnings
Geolocation and foot traffic	Mobile-device location pings at store or venue locations	Foot-traffic proxies for retail, dining, and travel
Shipping and logistics	Vessel-tracking (AIS) data, port congestion, rail and truck telemetry	Commodity flow and trade-volume forecasting
Social and search data	Social-media posts, review platforms, search-trend indices	Consumer-sentiment and brand-momentum signals
On-chain (blockchain) data	Public transaction ledgers, wallet flows, exchange inflows and outflows, stablecoin issuance	Crypto asset-flow signals, exchange-risk monitoring, AML and fraud forensics

Table 1.1: Major categories of alternative data

Two properties distinguish alternative data from the data types earlier chapters used. First, it usually arrives already partially processed by a vendor, a satellite imagery provider sells car counts, not raw pixels, so the modeler inherits whatever assumptions and errors are baked into that proprietary processing step, typically without visibility into it. Second, it is available at a frequency and granularity, individual stores, individual weeks, that traditional fundamental data (quarterly, firm-level) simply does not offer, which is precisely why it is valuable: it lets an analyst estimate a firm's performance before the firm itself reports it.

The table's last category deserves its own note, because it inverts both of those properties. **On-chain data**, the complete public transaction ledger a blockchain such as Bitcoin's or Ethereum's maintains by construction, records every transfer, wallet balance, and smart-contract interaction, free of charge and with

no vendor standing between the analyst and the raw record. Funds trading crypto assets themselves (Chapter 13's tokenization discussion introduces the instruments) read signals directly from it, net flows into and out of exchange-controlled wallets as a proxy for imminent selling or accumulation, stablecoin issuance as a proxy for capital entering the ecosystem, while risk and compliance teams watch the same ledger for a different reason, monitoring exchange solvency and tracing stolen or laundered funds. What the analyst inherits instead of a vendor's opaque preprocessing is pseudonymity: every party is an unlabeled address, so the commercial value-add in this category is entity labeling and address clustering, the same graph techniques Chapter 15 applies to shell-company layering chains, sold by the same analytics vendors compliance teams already rely on. That makes on-chain data the rare category where the raw data is free and complete while the interpretive layer is the scarce, priced asset, the exact reverse of a satellite feed, whose pixels are expensive and whose car counts are the deliverable.

1.2.2 The Alternative-Data Vendor Ecosystem

Unlike the market and fundamental data of Chapters 2 through 5, which arrive through well-established exchange feeds and regulatory filings, alternative data is procured through a comparatively young and heterogeneous vendor market, and how a fund evaluates and integrates a vendor matters almost as much as the underlying data source itself. A typical procurement process runs through several stages.

First, a fund identifies a candidate hypothesis, does satellite-derived parking-lot data actually predict Cascade's revenue, and requests a trial dataset covering a historical period the vendor already has on file. Second, the fund backtests that trial data against known, already-reported outcomes, the exercise Section 1.2.3 works through by hand, before paying for a live subscription; a vendor unwilling to provide a meaningful historical trial, or one whose trial-period performance does not hold up when checked, is a warning sign rather than a mere inconvenience. Third, if the trial is convincing, the fund negotiates a licensing agreement that specifies not just price but exclusivity (is this data available to other subscribers, and how many), delivery latency (Section 1.3.4 returns to why this matters enormously), and historical backfill (how far back the vendor can provide data, which determines how much history is available for the fund's own model validation).

This procurement cycle is itself a source of adverse selection worth naming directly: a vendor has every incentive to showcase the historical period and the subset of covered firms where its data performed best, since that is what sells subscriptions, so a trial dataset's strong backtested performance is not automatically evidence that the data will perform as well in the future or across firms the trial period did not happen to include. This is a version of the multiple-testing concern Chapter 6 raised about a researcher's own model search, now applied to a vendor's marketing rather than a researcher's specification search.

A subscription decision ultimately comes down to a cost-benefit calculation, whether the expected trading edge, scaled to the capital actually deployed on it, exceeds the data's licensing cost. Suppose a fund would deploy a \$50 million position whenever Section 1.2.4's signal fires long, which happened in five of the eight quarters tabulated in Table 1.3 below. Suppose too that the signal's incremental edge over an unconditional baseline is the 0.54 percentage points computed there, the 0.88% average return on signaled quarters minus the 0.34% average return across all quarters. With four earnings releases per year and a $5/8$ signal frequency, the fund expects roughly $4 \times 5/8 = 2.5$ signaled events annually, for an expected incremental profit of $\$50\text{M} \times 0.54\% \times 2.5 \approx \$678{,}000$ per year. Against Section 1.5's subscription-cost range, a satellite feed at \$250,000 per year plus a smaller web and card-panel bundle at \$150,000 per year, total data costs come to \$400,000. The net expected benefit is therefore roughly \$278,000 per year: a genuine but modest edge once data costs are netted out, and one that Section 1.3.4 warns will shrink further as more funds license the same feeds and compete the edge away.

This same arithmetic underlies a build-versus-buy decision many larger funds eventually face. Once expected trading volume across many similar signals is large enough, building an in-house satellite-analytics or web-scraping team can be cheaper in the long run than paying a vendor's margin on top of the same underlying imagery or logs. The cost is taking on the computer-vision and data-engineering work Section 1.3.3 describes directly, rather than buying an already-processed feature. Smaller funds typically lack the scale to justify this fixed cost and remain vendor customers indefinitely. That is one more way, alongside Section 1.5's subscription-cost figures, in which alternative data's advantages concentrate among larger, better-resourced market participants.

1.2.3 A Worked Example: Satellite Imagery and Retailer Revenue

Suppose a hypothetical home-improvement retailer, Cascade Home & Garden, is followed by a data vendor that counts cars in a representative sample of its store parking lots from satellite imagery, in the manner Section 1.5 describes being done for real retailers since 2010. Table 1.2 lists eight quarters of year-over-year car-count growth alongside Cascade's subsequently reported year-over-year revenue growth.

Quarter	Car-count growth (%)	Revenue growth (%)
Q1	2.1	1.5
Q2	−1.5	−0.3
Q3	4.8	4.5
Q4	0.6	1.0
Q5	6.2	4.0
Q6	−3.0	−1.0
Q7	3.5	3.8
Q8	1.0	−0.2

Table 1.2: Satellite car-count growth and Cascade's reported revenue growth, by quarter

Fitting the simple regression Revenue growth $= \alpha + \beta\,$(Car-count growth) by ordinary least squares, following Chapter 6's Section 6.3.1 method, gives $\bar{x} = 1.7125$, $\bar{y} = 1.6625$, $S_{xy} = \sum(x_i - \bar{x})(y_i - \bar{y}) = 34.09625$, $S_{xx} = \sum(x_i - \bar{x})^2 = 52.24875$, so $\hat{\beta} = S_{xy}/S_{xx} = 0.6528$ and $\hat{\alpha} = \bar{y} - \hat{\beta}\bar{x} = 0.5446$.

The fitted line, $\widehat{\text{Revenue growth}} = 0.5446 + 0.6528\,$(Car-count growth), achieves $R^2 = 0.870$, car-count growth alone explains 87% of the quarter-to-quarter variation in Cascade's revenue growth in this sample, genuinely strong for a single proxy variable, but, consistent with this book's practice of not overstating a model's fit, still leaves 13% of the variation unexplained.

Suppose satellite data for a ninth quarter arrives showing 5.0% car-count growth, three weeks before Cascade's earnings release. The regression predicts $\widehat{\text{Revenue growth}} = 0.5446 + 0.6528(5.0) = 3.809\%$, a concrete, tradeable estimate of a number the market will not see officially for three more weeks.

1.2.4 Combining Alternative Data Sources: A Multi-Source Earnings Signal

A single alternative data source is rarely used alone in practice; a real alternative-data desk combines several sources into one model, both to average out any single source's idiosyncratic noise and to capture aspects of a firm's business a single proxy misses. Figure 1.1 sketches the resulting pipeline.

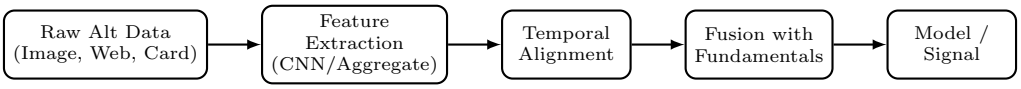

Figure 1.1: The alternative-data pipeline this chapter develops

Extending Section 1.2.3's example, suppose Cascade is also tracked by a web-traffic vendor (year-over-year growth in visits to Cascade's e-commerce site) and a card-panel vendor (year-over-year growth in aggregated card spending at Cascade's stores). Table 1.3 lists all three alternative-data growth rates alongside the resulting earnings surprise, actual revenue growth minus the analyst consensus estimate available at the time, and Cascade's stock return on the earnings-announcement day.

Regressing surprise on the three alternative-data growth rates by ordinary least squares, using the normal equations

$$\hat{\beta} = (X^\top X)^{-1} X^\top y,$$

with X's columns an intercept, car-count growth, web-traffic growth, and card growth, gives

$$\hat{\beta} = (-0.291,\ 1.092,\ -0.113,\ -0.792), \qquad R^2 = 0.837.$$

Quarter	Car (%)	Web (%)	Card (%)	Surprise (%)	Fitted (%)	Signal	Return (%)
Q1	2.1	3.0	1.5	0.5	0.48	Long	1.1
Q2	−1.5	−0.5	−1.0	−1.3	−1.08	Flat	−0.6
Q3	4.8	5.5	3.0	2.5	1.95	Long	2.0
Q4	0.6	1.5	0.8	0.0	−0.44	Flat	0.3
Q5	6.2	4.0	4.5	2.0	2.46	Long	−0.4
Q6	−3.0	−2.0	−1.8	−1.5	−1.92	Flat	−1.4
Q7	3.5	2.5	2.0	2.3	1.66	Long	1.5
Q8	1.0	2.0	0.5	−0.3	0.18	Long	0.2

Table 1.3: Multi-source alternative data, earnings surprise, and next-day return, by quarter

The negative coefficient on card growth is not a mistake: with car-count and web-traffic growth already in the regression, card growth's own explanatory power is largely redundant with theirs, and its small residual coefficient partly offsets collinear noise the other two variables share, a common occurrence when several correlated proxies for the same underlying quantity are entered into one regression together, and a reminder that individual coefficients in a multi-source model should not be read as clean, isolated effects the way Chapter 6's Section 6.3.1 single-variable coefficient can be.

Column "Fitted" reports each quarter's fitted surprise from this regression, and column "Signal" converts a positive fitted surprise into a long position taken ahead of the earnings release (flat otherwise), the same logic Chapter 16's Section 16.3.1 applied to a sentiment score. Five of the eight quarters signal long; of those, four (Q1, Q3, Q7, Q8) are followed by a positive next-day return and one (Q5) by a negative return despite a strongly positive fitted surprise, an 80% hit rate.

The signaled quarters average a 0.88% next-day return versus 0.34% across all eight quarters, a real edge, but, exactly as Chapter 6 and Chapter 16's Section 16.3.1 both emphasize, a probabilistic one: Q5's miss shows that even a well-fitted, multi-source alternative-data model does not eliminate the risk that a single quarter's price reaction depends on something the data could not capture, guidance issued alongside the earnings release, for instance. A ninth quarter with car, web, and card growth of 5.0%, 4.5%, and 3.5% respectively yields a fitted surprise of

$$-0.291 + 1.092(5.0) - 0.113(4.5) - 0.792(3.5) = 1.888\%,$$

again a tradeable estimate available before the official release.

1.3 Working with Alternative Data

Alternative data breaks alignment, scale, and quality assumptions that price data never did.

1.3.1 Point-in-Time Alignment and the Cost of Look-Ahead Bias

Section 1.2.4's regression silently assumed that all three growth rates for a given quarter were genuinely available before that quarter's earnings release. In reality, a card-panel vendor's data is typically delayed: the panel for a given quarter is often not fully assembled and delivered to subscribers until well after that quarter's earnings have already been reported, because the underlying card issuers themselves need time to compile and anonymize transaction records. A backtest that uses the current quarter's card figure as if it had been available in time, when in fact only the prior quarter's card figure would genuinely have been in hand, is committing look-ahead bias, using information the backtest could not actually have had.

To see how much this matters, refit Section 1.2.4's regression on quarters Q2 through Q8 (seven quarters, dropping Q1 since it has no prior quarter to lag from) two ways. The naive version uses each quarter's own, contemporaneous card growth, exactly as before:

$$\hat{\beta}_{\text{naive}} = (-0.290,\ 1.102,\ -0.119,\ -0.802), \qquad R^2 = 0.837,$$

with four quarters signaling long (Q3, Q5, Q7, Q8) and a 75% hit rate. The realistic version instead uses, for each quarter, the card growth reported one quarter earlier, the figure that would genuinely have been available before that quarter's earnings release:

$$\hat{\beta}_{\text{realistic}} = (0.251,\ 0.424,\ -0.100,\ -0.286), \qquad R^2 = 0.873,$$

but Q8 no longer signals long once the lag is respected, leaving only three quarters (Q3, Q5, Q7) with a 66.7% hit rate, and a higher average return (1.03% versus 0.83%) on that smaller, more selective set of trades.

The lesson is not that the lagged, realistic version is simply worse, its R^2 is actually higher and its average signaled return is larger, but that it is a *different* model making *different* trading decisions than the naive version, on the very same underlying quarters. A backtest run on the naive, contemporaneous-card version would report performance for trades the fund could never actually have placed, since the card data those trades depend on was not yet in hand, regardless of whether that misspecified backtest happens to look better or worse than the honest one. This is why Section 1.3.4 treats delivery lag as a data-quality issue rather than a mere modeling nuance: the direction of the resulting bias is not predictable in general, but the fact that ignoring real-world delivery timing invalidates the backtest as a measure of tradeable performance is completely general, and point-in-time data discipline, using, for every historical date, only the data vintage that would genuinely have been available on that date, is the standard defense against it.

1.3.2 Feature Engineering for Alternative Data

Raw alternative-data readings, like the raw market and fundamental data of Chapters 2 through 5, are rarely used exactly as delivered; Section 1.3.1 addressed when a reading becomes usable, and this section addresses what should be done to it before it is usable at all.

Seasonal adjustment. Web traffic, foot traffic, and card-transaction volumes all carry strong calendar effects, day-of-week patterns, holiday spikes, and back-to-school or year-end shopping seasons, that have nothing to do with a firm's underlying performance. Comparing a raw reading against the same period one year earlier, exactly what year-over-year growth rates such as Section 1.2.3's car-count growth already do, removes most of this seasonality automatically, provided the calendar aligns cleanly (a retailer's Thanksgiving week, for instance, falls on a different date each year, which can distort a naive year-over-year comparison unless the alignment is done by trading day or holiday-relative position rather than by calendar date).

Peer-relative normalization. A retailer's car-count or web-traffic growth reflects both firm-specific performance and sector-wide conditions, gasoline prices, general consumer spending, weather, that affect every retailer in the sector simultaneously. Subtracting a sector-average benchmark from a firm's own reading isolates the firm-specific component a stock-picking strategy actually cares about.

Suppose sector-average car-count growth for Cascade's peer group across the same eight quarters of Table 1.2 is $(1.0, -0.5, 2.0, 0.3, 2.5, -1.0, 1.5, 0.5)$; subtracting it from Cascade's own car-count growth gives a peer-adjusted series $(1.1, -1.0, 2.8, 0.3, 3.7, -2.0, 2.0, 0.5)$. Refitting Section 1.2.3's regression with this peer-adjusted series as the predictor gives $\hat{\beta} = 1.064$, $\hat{\alpha} = 0.678$, and $R^2 = 0.867$, essentially unchanged from the raw series' $R^2 = 0.870$. In this particular sample sector-wide movements were fairly small relative to Cascade's own idiosyncratic variation, so peer-adjustment does not change the fit much here, but the adjustment matters far more in a period when the whole sector moves together, a broad consumer-spending downturn, a fuel-price spike depressing driving-based foot traffic sector-wide, where an un-adjusted signal would mistake a sector-wide move for firm-specific news.

1.3.3 Neural Networks for Structured and Unstructured Data

Section 1.2.3 treated the vendor's car count as a finished number, but the vendor itself had to produce that count from raw satellite pixels, and Section 1.2.4's web-traffic and card figures likewise start from raw logs or transaction records. Chapter 6 introduced the feedforward network, in which every input feeds every hidden unit with its own independent weight; that architecture ignores two kinds of structure alternative data routinely has, the two-dimensional grid structure of an image and the ordered, sequential structure of a time-stamped log, and specialized architectures were developed to exploit each, alongside a third architecture that needs no labeled target at all.

Convolutional Neural Networks for Grid-Structured Data

Suppose a vendor wants to teach a model to count cars automatically in a million-pixel satellite tile rather than paying an analyst to do it by eye. A feedforward network, of the kind Chapter 6 covers, could in principle treat every one of those million pixels as an independent input feature, but it would need a separate learned weight connecting each pixel to each hidden unit, and it would have to relearn from scratch that a cluster of bright pixels in the top-left corner and the identical cluster in the bottom-right corner both mean the same thing, a car, since nothing in a feedforward network's structure tells it that a pattern's meaning does not depend on where in the image it sits.

A convolutional neural network (CNN) exploits this redundancy: a useful pattern in an image, a car's outline, an edge, a texture, can appear anywhere in the frame, so instead of learning a separate weight for every pixel-hidden-unit pair, a CNN learns a small filter (or kernel) and slides it across the entire image, reusing the same weights at every position. This weight sharing is what makes CNNs practical for images: a 1000×1000-pixel satellite tile has a million pixels, far too many for a feedforward network's per-pixel weights to be learned from any realistic training set, but a 2×2 filter has only four weights to learn regardless of image size.

Consider a stylized 4×4 satellite image patch of a single parking lot, with pixel intensities between 0 (dark asphalt) and 1 (a car's bright roof), containing two clusters of parked cars in opposite corners:

$$I = \begin{pmatrix} 0.1 & 0.1 & 0.8 & 0.9 \\ 0.1 & 0.1 & 0.9 & 0.8 \\ 0.7 & 0.8 & 0.1 & 0.1 \\ 0.8 & 0.9 & 0.1 & 0.1 \end{pmatrix}.$$

Convolving this image with a 2×2 averaging filter

$$K = \begin{pmatrix} 0.25 & 0.25 \\ 0.25 & 0.25 \end{pmatrix}$$

at stride 1 (sliding the filter one pixel at a time, computing the elementwise product with each 2×2 patch and summing) produces a 3×3 feature map. The top-left entry, for instance, averages the patch

$$\begin{pmatrix} 0.1 & 0.1 \\ 0.1 & 0.1 \end{pmatrix}$$

to 0.1; the entry one column to the right averages

$$\begin{pmatrix} 0.1 & 0.8 \\ 0.1 & 0.9 \end{pmatrix}$$

to $(0.1 + 0.8 + 0.1 + 0.9)/4 = 0.475$. Carrying this out for all nine positions gives

$$\text{Feature map} = \begin{pmatrix} 0.100 & 0.475 & 0.850 \\ 0.425 & 0.475 & 0.475 \\ 0.800 & 0.475 & 0.100 \end{pmatrix}.$$

The two corner entries where a car cluster sits, 0.850 (top right) and 0.800 (bottom left), stand out clearly above the two empty corners, both 0.100, and above the mixed-boundary middle entries, which is exactly what this filter is designed to do: highlight regions of concentrated brightness.

A real, trained CNN learns many such filters simultaneously (edge detectors, corner detectors, texture detectors), stacks several convolutional layers so that later layers respond to increasingly complex combinations of earlier layers' features, and typically applies a ReLU activation and a pooling step (taking, say, the maximum of each 2×2 block of the feature map) after each convolution, reducing the map's size while keeping its strongest responses, before finally feeding the result into feedforward layers of the kind Chapter 6 described to produce a single number, an estimated car count.

Stacking convolutional layers naively, however, runs into a practical training problem once a network gets deep: gradients computed by backpropagation (Chapter 6) must flow back through every layer, and in

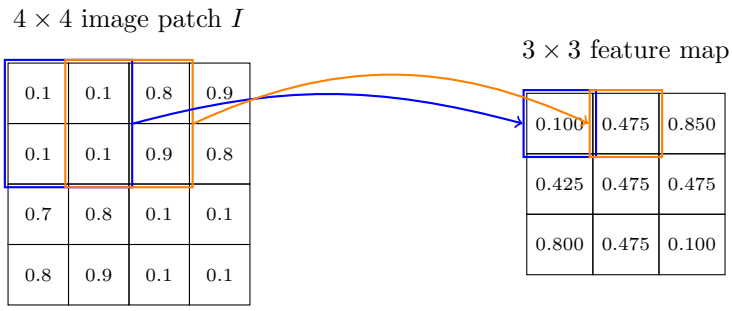

Figure 1.2: Sliding a 2×2 averaging filter across the 4×4 satellite patch I at stride 1

a sufficiently deep plain CNN they can shrink toward zero long before reaching the earliest layers, leaving those layers essentially untrained, the vanishing-gradient problem.

The architecture that made very deep CNNs practical, the residual network (ResNet), addresses this with a simple structural change: each block adds a shortcut connection carrying the block's input directly to its output, so the block only has to learn a residual adjustment on top of what already passed through unchanged, rather than having to learn the entire transformation from scratch, letting gradients flow through the shortcut even when a particular block's own learned transformation contributes little. This is the same practical instinct as Chapter 12's shrinkage-based portfolio methods, do not force a model to learn from nothing what could instead be inherited or only mildly adjusted, applied to network depth rather than portfolio weights.

Recurrent Neural Networks for Sequential Data

Suppose an analyst wants to judge whether Cascade's web traffic is accelerating going into its next earnings release using the last several weeks of readings rather than just the most recent one, since a single week's number cannot distinguish a steady climb from a spike that is about to reverse. A feedforward network could take all several weeks as separate input features, but it would treat them as an unordered bag, with nothing in its structure telling it that the reading from three weeks ago came before the reading from last week.

A recurrent neural network (RNN) addresses this: an ordered sequence, such as several successive weeks of web-traffic readings, in which the order itself carries information a feedforward network, which treats its inputs as an unordered set of features, would discard. An RNN processes a sequence one step at a time, maintaining a hidden state h_t that is passed forward and updated at every step,

$$h_t = \tanh(w_x x_t + w_h h_{t-1} + b),$$

so that h_t is, in principle, a function of every input the network has seen so far, $x_1, \ldots, x_t$, not just the current one, giving the network a form of memory that Chapter 7's autoregressive and GARCH recursions share in spirit, each also defines a current value as a function of a prior state plus new information, but that Chapter 6's feedforward network has no analogue for.

Suppose Cascade's e-commerce site shows three successive weekly readings of normalized web-traffic growth in the final three weeks before its next earnings release, $x_1 = 0.5$, $x_2 = -0.2$, $x_3 = 0.9$, and a single-hidden-unit RNN with weights $w_x = 0.6$, $w_h = 0.4$, $b = 0.1$, and initial hidden state $h_0 = 0$. Step by step:

$$\begin{aligned} h_1 &= \tanh(0.6 \times 0.5 + 0.4 \times 0 + 0.1) = \tanh(0.400) = 0.3799, \\ h_2 &= \tanh(0.6 \times (-0.2) + 0.4 \times 0.3799 + 0.1) = \tanh(0.13196) = 0.1312, \\ h_3 &= \tanh(0.6 \times 0.9 + 0.4 \times 0.1312 + 0.1) = \tanh(0.69248) = 0.5996. \end{aligned}$$

Notice that h_2 is small, close to zero, not because x_2 was extreme (it was mildly negative) but because it also carries a damped memory of h_1; the hidden state genuinely blends past and present rather than reacting to the latest input alone.

Passing the final hidden state through an output layer, $\hat{y} = \sigma(w_y h_3 + b_y)$, where $\sigma(z) = 1/(1+e^{-z})$ is Chapter 6's sigmoid, with $w_y = 1.2$ and $b_y = -0.3$, gives $\hat{y} = \sigma(1.2 \times 0.5996 - 0.3) = \sigma(0.4195) = 0.6034$, which might, for instance, be interpreted as a 60.3% estimated probability that Cascade's upcoming quarter shows accelerating (rather than decelerating) revenue growth. Figure 1.3 unrolls this same computation across its three time steps, making explicit how each hidden state depends on both the current input and the previous hidden state.

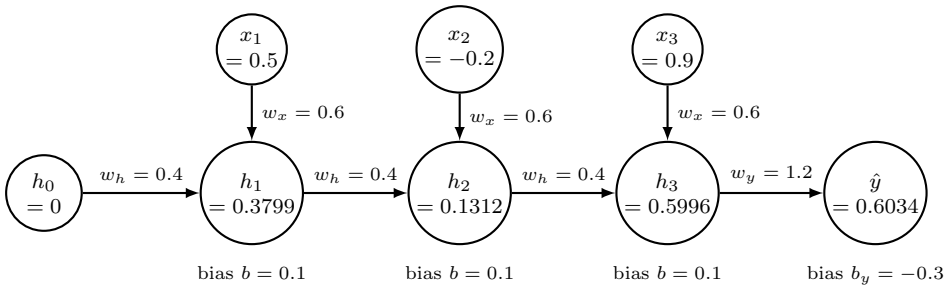

Figure 1.3: The single-hidden-unit RNN unrolled across its three time steps

In practice, a plain RNN's memory decays quickly over long sequences, since each step multiplies the running influence of an early input by another factor less than one, a limitation addressed by the long short-term memory (LSTM) architecture. An LSTM maintains a separate cell state c_t, alongside the hidden state, updated as

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t,$$

where

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

is a forget gate controlling how much of the previous cell state to retain,

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$

is an input gate controlling how much of a new candidate value

$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

to admit, and $\odot$ denotes elementwise multiplication; a third, output gate

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$

then controls how much of c_t is exposed as the hidden state

$$h_t = o_t \odot \tanh(c_t).$$

The three gates use σ, whose output lies in $(0, 1)$ and so reads as a fraction retained or admitted, while the candidate uses tanh, whose output lies in $(-1, 1)$ and so carries a signed value to be written into the cell state. The worked example below turns on precisely that difference: from identical arguments it computes $i_1 = \sigma(0.25) = 0.5622$ but $\tilde{c}_1 = \tanh(0.25) = 0.2449$. Figure 1.4 puts these six equations in one picture, with the cell state running along the top and the gates feeding it from below.

Because the cell state's update is additive rather than the plain RNN's fully multiplicative recursion, a forget gate near 1 lets information pass across many more time steps largely undamped, directly addressing the memory-decay limitation the single-unit example above illustrates, at the cost of three times as many weights to learn as the plain recurrent unit.

A worked numerical example makes this concrete rather than leaving it as a qualitative claim. Applying a single-unit LSTM to the identical three-week input sequence used for the plain RNN above ($x_1 = 0.5$, $x_2 = -0.2$, $x_3 = 0.9$, $h_0 = 0$, $c_0 = 0$), with illustrative gate weights $w_{f,h} = 0.5$, $w_{f,x} = 0.3$, $b_f = 2.0$ (a

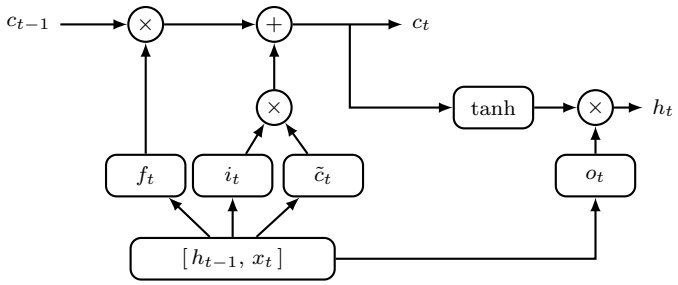

Figure 1.4: The LSTM cell. The cell state crosses the top of the diagram scaled once and added to once, never passed through a squashing function, which is why information survives along it; the three gates below all read the same concatenated $[h_{t-1}, x_t]$.

deliberately large bias, chosen so the forget gate stays close to 1 and the memory-retention effect is visible) and $w_{i,h} = w_{i,x} = w_{c,h} = w_{c,x} = w_{o,h} = w_{o,x} = 0.5$ with all other biases at 0, the first step gives

$$f_1 = \sigma(2.15) = 0.8957, \qquad i_1 = \sigma(0.25) = 0.5622, \qquad \tilde{c}_1 = \tanh(0.25) = 0.2449,$$
$$c_1 = f_1 c_0 + i_1 \tilde{c}_1 = 0.1377, \qquad o_1 = \sigma(0.25) = 0.5622, \qquad h_1 = o_1 \tanh(c_1) = 0.0769.$$

Continuing this recursion through the second and third steps gives

$$f_2 = 0.8785, \qquad c_2 = 0.0912, \qquad h_2 = 0.0441,$$
$$f_3 = 0.9082, \qquad c_3 = 0.3537, \qquad h_3 = 0.2092;$$

passing h_3 through the same output layer as the plain RNN example ($w_y = 1.2$, $b_y = -0.3$) gives $\hat{y} = \sigma(1.2 \times 0.2092 - 0.3) = 0.4878$.

The forget gates $f_2 = 0.8785$ and $f_3 = 0.9082$ are the mechanism worth isolating: their product, $0.8785 \times 0.9082 \approx 0.7979$, is the fraction of the original cell state c_1 that survives, purely through the forget gate's own multiplicative decay, two additional time steps later, versus only $w_h^2 = 0.4^2 = 0.16$ for the plain RNN's equivalent two-step decay of its hidden state's direct dependence on h_1.

Roughly 80% of c_1's information survives to c_3 in the LSTM, against roughly 16% of the plain RNN's analogous quantity, a concrete, order-of-magnitude illustration of the memory-decay fix the forget gate is designed to provide: not that information never decays in an LSTM, the forget gate is still strictly less than 1, but that a well-trained forget gate can be pushed close enough to 1 that memory persists over many more steps than a plain RNN's fixed recurrent weight allows. Figure 1.5 draws the first step of this same computation as a cell diagram, showing all four gates computed from the identical inputs before they combine into the new cell and hidden states.

Autoencoders for Unsupervised Anomaly Detection

Suppose a data-quality analyst wants to flag whenever a quarter's alternative-data readings look unusual, without knowing in advance what "unusual" means and without a single labeled historical example of a bad reading to train against, an unsupervised problem the CNN and RNN examples above cannot address, since both are supervised, trained toward a known target, an estimated car count, a probability of accelerating growth.

An autoencoder is trained without any label at all: it learns an encoder that compresses an input down to a small bottleneck representation and a decoder that reconstructs the original input from that bottleneck, with the reconstruction error itself as the only training signal. Once trained on typical, well-behaved data, an autoencoder reconstructs anomalous inputs poorly, since its bottleneck was never shaped to represent whatever made them unusual, which makes reconstruction error a natural anomaly score, extending Chapter 15's classification-based anomaly detection to a setting with no labeled examples of what an anomaly looks like.

Consider a simple linear autoencoder, encoder weights $w = (0.5, 0.3, 0.2)$ applied to Table 1.3's three alternative-data growth rates $x = (\text{car}, \text{web}, \text{card})$ for each quarter, producing a single bottleneck code

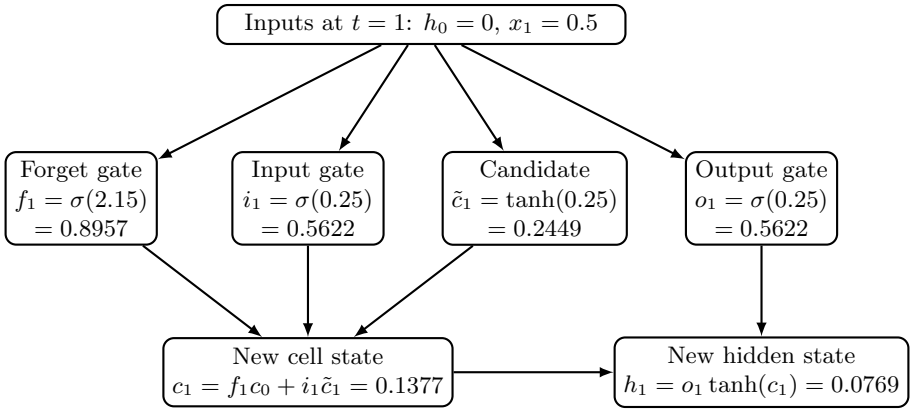

Figure 1.5: The LSTM cell's first time step, computed from the worked example in the text

$z = w \cdot x$, and a tied-weight decoder that reconstructs $\hat{x} = z\,w$. For Q4, $x = (0.6, 1.5, 0.8)$, so $z = 0.5(0.6)+0.3(1.5)+0.2(0.8) = 0.91$, and $\hat{x} = 0.91(0.5, 0.3, 0.2) = (0.455, 0.273, 0.182)$, giving a reconstruction error $\sum(x_i - \hat{x}_i)^2 = (0.6 - 0.455)^2 + (1.5 - 0.273)^2 + (0.8 - 0.182)^2 = 1.909$.

Carrying this out for all eight quarters and expressing each quarter's error relative to its own total energy $\sum x_i^2$ (so that quarters with naturally larger growth rates are not automatically flagged just for being larger) gives relative errors ranging from 39.5% (Q7) to 58.7% (Q4). Q4 stands out as the quarter whose alternative-data pattern the autoencoder reconstructs worst, not because its numbers are the largest in absolute terms, Q5's raw error of 30.9 is far larger, but because Q4's web-traffic growth (1.5%) is unusually large relative to its car-count and card growth (0.6% and 0.8%), a pattern this particular bottleneck direction does not represent well.

In practice, this would be the kind of quarter a data-quality analyst should investigate first, is the web-traffic reading itself an error, a one-time promotional spike, or a genuine early signal the other two sources have not yet picked up, before that quarter's data is trusted in a downstream model.

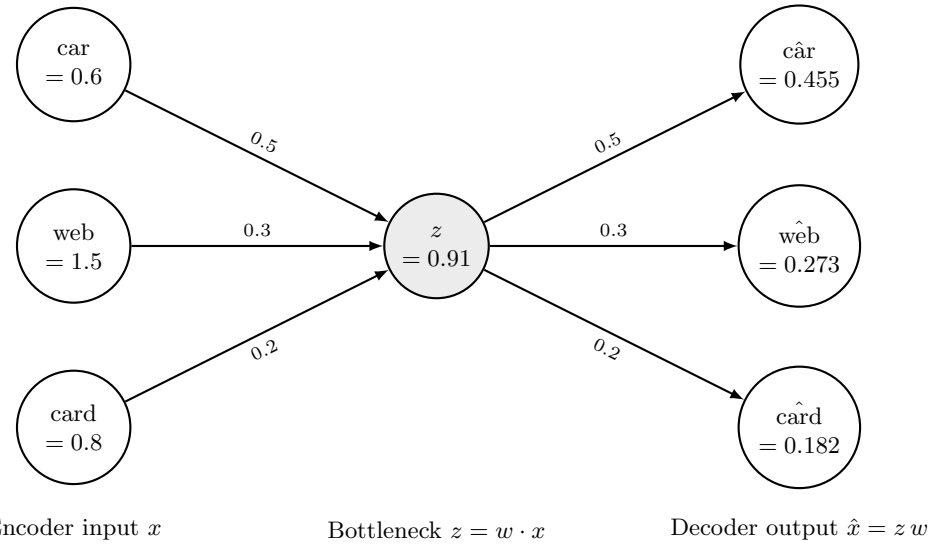

Figure 1.6: The linear autoencoder applied to Q4's three alternative-data growth rates

Transformers as a Unifying Architecture

Chapter 16 developed the transformer architecture and its self-attention mechanism (Section 16.4.1) for text, where each token attends to every other token in a sequence. The same mechanism generalizes beyond text: a vision transformer (ViT) divides an image into fixed-size patches, treats each patch the way a transformer treats a word token, and applies self-attention across patches instead of convolution across pixels, letting a satellite image classifier weigh a distant part of the image against the current patch directly, something a CNN's local filters only do indirectly, through many stacked layers.

The companion notebook makes this concrete by splitting the same 4×4 satellite image the CNN example convolved into four non-overlapping 2×2 patches, treating each as a token exactly as Chapter 16 treated a word, and computing scaled dot-product self-attention across them. Querying from patch 2 (the top-right car cluster, mean brightness 0.85) against keys built from each patch's own brightness, patch 2 attends to itself with weight 0.6174 and to patch 3 (the bottom-left car cluster, mean brightness 0.80, diagonally the farthest patch in the image) with weight 0.3817, while the two empty patches 1 and 4 (mean brightness 0.10 each) receive a combined weight of only 0.001.

This is the claim above made numerical: patch 2's most relevant neighbor is patch 3, a content-similar patch on the opposite corner of the image, not either of the two empty patches nearer to it, and self-attention identifies that relevance in a single step rather than requiring several stacked convolutional layers to propagate information across that same distance. In current practice, CNNs remain common for smaller or highly structured image tasks (including much car-counting and object-detection work), while transformer-based architectures increasingly dominate both large-scale image classification and the web-traffic and card-transaction sequence modeling that once used RNNs, so that a single architectural family, self-attention, now spans text (Chapter 16), images, and sequences alike.

1.3.4 Data Quality, Bias, and Integration Challenges

Alternative data's advantages, frequency, granularity, and a genuine informational edge before official reporting, come with distinctive quality problems that do not have close analogues in the market and fundamental data earlier chapters used.

Coverage bias. A satellite vendor can only cover stores it has usable imagery for, typically larger, free-standing locations with visible parking lots, so its car-count sample systematically underrepresents smaller-format or mall-based stores, and a model trained only on the covered subset may not generalize to a retailer's business as a whole.

Vendor black-box processing. As Section 1.2.1 noted, a vendor typically sells an already-processed feature, a car count, a traffic index, not raw pixels or logs, so an analyst inherits whatever computer-vision or aggregation errors are baked into that pipeline without being able to inspect or correct them directly; a vendor's undisclosed methodology change between periods can look, in the data, indistinguishable from a genuine change in the underlying business.

Delivery lag and look-ahead bias. A backtest that assumes alternative data was available on the date it describes, rather than the (often later) date it was actually delivered to subscribers, silently uses information that would not have been tradeable in real time; Section 1.2.4's multi-source signal, for instance, is only genuinely useful if all three data sources reach a trading desk before the earnings release they are meant to anticipate, not merely before the quarter ends.

Licensing cost and alpha decay. High-quality alternative data is expensive, often licensed exclusively or semi-exclusively, and Section 1.5's case documents a genuine 4-5% informational edge around earnings from satellite car counts; as more funds license the same feed and trade on the same signal, that edge is competed away over time in the way Chapter 6 described a known profitable pattern eroding once it becomes widely traded, so an alternative-data edge, like any other, has a shelf life.

Privacy and legal exposure. Card-panel and geolocation data are aggregated and anonymized before sale, but reconstructing individual behavior from supposedly anonymized data has repeatedly proven possible in other domains, so a vendor's privacy practices, and a buyer's own legal exposure, are due-diligence questions that market and fundamental data never raised.

The line between alternative data and material non-public information is not always bright. Card-panel and web-traffic vendors aggregate across many individual cardholders or visitors specifically so that no single company's non-public information is being sold, but the boundary is a matter of legal judgment

rather than a fixed technical threshold, and a narrowly enough scoped feed, transaction data from only a handful of a firm's largest customers, say, can approach material non-public information even though it never touches the firm's own systems and no insider ever leaks anything, a genuinely new compliance risk that Chapter 15's insider-trading discussion, framed entirely around information leaking from inside a firm, did not anticipate.

1.4 Applications and Advanced Methods

This part builds a signal from disclosure text, then surveys the deep-learning methods the data invites.

1.4.1 Building an ESG Signal Directly from Disclosure Text

Chapter 12 showed that six major ESG rating providers agree with one another only weakly, an average pairwise correlation of just 38% to 71%, and traced most of that disagreement to providers measuring different underlying facts about a company and weighting them differently, not to noisy measurement of an otherwise agreed-upon concept. Section 1.2.2's build-versus-buy logic applies to this problem exactly as it applies to satellite imagery: rather than purchasing a vendor's already-computed ESG score and inheriting whatever undisclosed methodology produced it, a fund can build its own narrow, transparent environmental-tone signal directly from a company's public disclosures, using the same text-analysis tools Chapter 16 developed for a different purpose.

Suppose Meridian Fabrication, the manufacturer whose ratio analysis Chapter 4 worked through in detail, files a 10-K whose risk-factors section runs to several pages. Chapter 16's Naive Bayes classifier (Section 16.3.2) can be extended with a third training class, Environmental Risk, built from a small set of sentences using vocabulary (emissions, environmental, climate, remediation) that neither the original Credit Risk nor Market Risk class's training sentences ever mention. The retrained classifier then flags four sentences in Meridian's filing as Environmental Risk with high confidence, each keying on at least one environment-specific term the other two classes' training vocabulary lacks entirely:

> S1: "Management remains confident that planned facility upgrades will reduce our emissions, but continued regulatory headwinds could still increase compliance costs."
> S2: "We remain cautious about the pace of emissions reduction given persistent supply chain headwinds affecting our environmental capital projects."
> S3: "Emissions declined for the third consecutive year, reflecting strong progress on our environmental targets despite a challenging regulatory backdrop."
> S4: "Physical risks from climate change, including potential flooding at our coastal facility, remain a weak point in our disaster preparedness planning and a continuing concern for the board."

Applying Chapter 16's Loughran-McDonald dictionary tone score (Section 16.2.3) to each of these four flagged sentences, using the same small positive word set {strong, growth, exceeded, confident, improved} and negative word set {cautious, headwinds, decline, declined, weak, concern, concerns} Chapter 16 used to score its two earnings-call transcripts, gives

$$\begin{aligned}
&\text{S1: } (1-1)/20 = 0.0,\\
&\text{S2: } (0-2)/19 \approx -0.1053,\\
&\text{S3: } (1-1)/19 = 0.0,\\
&\text{S4: } (0-2)/28 \approx -0.0714,
\end{aligned}$$

an average environmental tone score of -0.0442 across the four sentences, a mildly cautious reading of Meridian's own environmental disclosure.

This exercise surfaces a genuinely new data-quality problem, distinct from anything Section 1.3.4 catalogued, that is specific to building ESG signals from generic financial-sentiment tools: the Loughran-McDonald lexicon was built to score revenue and earnings language, where a word like "declined" is reliably bad news, but the same word describes unambiguously good environmental news in sentence S3, emissions falling for a third consecutive year, and the lexicon cannot tell the difference. S3's tone score of 0.0 is,

if anything, understating how positive that sentence actually is; a lexicon purpose-built for environmental disclosure, distinguishing a decline in emissions from a decline in revenue, would be needed to fix this, and no such standardized, publicly available dictionary exists yet the way Loughran-McDonald's does for general financial tone.

Comparing this section's result to Chapter 12's worked example makes the deeper point. Chapter 12's three fictional assets were scored on a common, if arbitrary, 0-to-100 composite scale, and its worked example varied only the *weight* providers place on the same three E, S, and G sub-scores, one of three sources of divergence Berg, Kölbel, and Rigobon identify. This section's -0.0442 tone score is not on that scale at all, and is not directly comparable to it: it measures something narrower (the tone of language in one filing's risk-factors section) using an entirely different method (word counting rather than a proprietary composite index), which is the *scope* and *measurement* divergence the same paper identifies as the larger share of why ESG ratings disagree. Building an in-house, transparent signal like this one does not resolve the disagreement documented in Chapter 12; it adds a fourth, differently-scoped measurement to a field that, per that same evidence, already has too many mutually inconsistent ones.

What it buys instead is auditability: unlike a vendor's black-box score, every step from raw sentence to final number in this section can be inspected, recomputed, and defended to a model validator in the way Chapter 8's model-risk-management framework requires.

1.4.2 Advanced Topics: Transfer Learning, Graphs, and Fusion

Three further techniques extend the neural network toolkit of Section 1.3.3 in directions that matter increasingly in real alternative-data practice: reusing an already-trained model instead of training from scratch, exploiting network structure between entities rather than treating each firm in isolation, and combining several data modalities into one representation.

Transfer Learning: Fine-Tuning a Pretrained Vision Backbone

Section 1.3.4 noted that a newly available alternative-data feed typically has only a short history, too little, on its own, to train a large CNN's millions of weights from scratch. Transfer learning sidesteps this: a CNN already trained on a large, general image dataset (which has already learned generic filters for edges, textures, and shapes) is reused as a frozen feature extractor, and only a small final layer, mapping its output features to the task at hand, car count, cloud cover, construction status, is trained on the limited task-specific data available. Because the frozen backbone contributes no gradient updates, this final-layer training is a direct application of Chapter 6's gradient descent to far fewer parameters than training a full network would require.

Suppose a pretrained backbone, frozen, produces a two-dimensional feature vector $f = (0.7, 0.3)$ for a new satellite image patch, and the task is to fine-tune a final logistic layer, currently $w = (0.4, 0.4)$, $b = -0.1$, to predict whether the patch contains a car cluster (label $y = 1$ for this patch). The current prediction is

$$\hat{y} = \sigma(w \cdot f + b) = \sigma(0.4 \times 0.7 + 0.4 \times 0.3 - 0.1) = \sigma(0.30) = 0.5744.$$

Following Chapter 6's gradient descent update with learning rate $\eta = 0.5$, the gradient of the log-loss with respect to w is

$$(\hat{y} - y)f = (0.5744 - 1)(0.7, 0.3) = (-0.298, -0.128),$$

and with respect to b is $\hat{y} - y = -0.426$, giving updated weights

$$w \leftarrow (0.4, 0.4) - 0.5(-0.298, -0.128) = (0.549, 0.464)$$

and

$$b \leftarrow -0.1 - 0.5(-0.426) = 0.113.$$

Recomputing the prediction with these updated weights gives

$$\hat{y} = \sigma(0.549 \times 0.7 + 0.464 \times 0.3 + 0.113) = \sigma(0.636) = 0.6539,$$

closer to the true label of 1 after a single gradient step on a handful of parameters, illustrating why transfer learning lets a fund with only a few hundred labeled satellite patches still fine-tune a usable classifier, rather than needing the millions of labeled images a full CNN would require if trained from scratch.

Graph Neural Networks for Ownership and Counterparty Networks

Chapter 15 modeled a shell-company layering network as a static sequence of hops, one entity moving funds to the next. A graph neural network (GNN) treats such a network as a live computational structure: each node's representation is updated by aggregating its neighbors' representations, so that risk (or any other property) genuinely propagates through the network rather than being assessed for each entity in isolation, the structure alternative data increasingly captures, common addresses, shared directors, correlated transaction timing, that a purely tabular model discards. A single GNN layer updates node v's representation as

$$h_v = \sigma(w_1 x_v + w_2 \cdot \text{mean}_{u \in N(v)}(x_u) + b),$$

where $N(v)$ is the set of v's neighbors in the graph.

Consider a four-node layering chain, $A \to B \to C \to D$, extending Chapter 15's shell-company example, with initial suspicion scores $x_A = 0.9$ (a flagged placement account), $x_B = 0.2$, $x_C = 0.1$, and $x_D = 0.05$ (an apparently unremarkable integration account), and neighbor sets

$$N(A) = \{B\},\ N(B) = \{A, C\},\ N(C) = \{B, D\},\ N(D) = \{C\}.$$

With weights $w_1 = 0.6$, $w_2 = 0.8$, $b = -0.05$, one round of mean-aggregation gives

$$h_B = \sigma\big(0.6(0.2) + 0.8 \cdot \tfrac{0.9+0.1}{2} - 0.05\big) = \sigma(0.47) = 0.6154$$

and

$$h_C = \sigma\big(0.6(0.1) + 0.8 \cdot \tfrac{0.2+0.05}{2} - 0.05\big) = \sigma(0.11) = 0.5275,$$

compared to $h_A = \sigma(0.65) = 0.6570$ and $h_D = \sigma(0.06) = 0.5150$. Node B's updated score, 0.615, is far higher than its own raw feature of 0.2 would suggest, purely because it is one hop from the flagged account A, and even node C, two hops away, shows a modest elevation to 0.528 from a raw 0.1; a model that scored each account independently, the way Chapter 15's original logistic regression did, would miss this kind of network-propagated risk.

Multi-Modal Fusion via Cross-Attention

A single financial decision often draws on several alternative-data modalities at once, an image feature from Section 1.3.3's CNN, a sentiment embedding from Chapter 16's text pipeline, and a tabular alternative-data summary like Section 1.2.4's regression inputs, and combining them well matters as much as extracting each one individually. Cross-attention, a direct extension of Chapter 16's self-attention mechanism (Section 16.4.1), treats one modality's representation as the query and the other modalities as key-value pairs to attend over:

$$\text{Attention}(Q, K, V) = \text{softmax}(QK^\top / \sqrt{d_k})V,$$

exactly as before, but now fusing modalities instead of fusing word tokens.

Suppose a tabular query vector $Q = (0.6, 0.8)$ (summarizing Cascade's alternative-data growth direction) attends over an image modality with key $K_{\text{img}} = (0.9, 0.1)$ and value $V_{\text{img}} = (0.7, 0.2)$, and a text-sentiment modality with key $K_{\text{txt}} = (0.2, 0.9)$ and value $V_{\text{txt}} = (-0.3, 0.6)$ (a mildly cautious sentiment reading). With $d_k = 2$, the scaled dot-product scores are

$$Q \cdot K_{\text{img}} / \sqrt{2} = 0.62 / 1.4142 = 0.4385$$

and

$$Q \cdot K_{\text{txt}} / \sqrt{2} = 0.84 / 1.4142 = 0.5941;$$

applying softmax gives attention weights 0.4612 on the image modality and 0.5388 on the text modality. The fused representation is

$$0.4612(0.7, 0.2) + 0.5388(-0.3, 0.6) = (0.161, 0.416),$$

a single vector that leans slightly more on the cautious text signal than on the image signal, and that a downstream model, a trading rule in the spirit of Section 1.2.4, a credit decision, a fraud flag, can consume in place of either modality alone, without the analyst having to hand-specify how much weight each modality deserves.

Self-Supervised Contrastive Pretraining

Section 1.4.2's transfer-learning example assumed a pretrained backbone was already available; contrastive pretraining is one common way such a backbone gets trained in the first place, without any human-provided labels at all, which matters enormously for satellite imagery, where labeled examples (this patch definitely contains three cars) are far scarcer than the raw imagery itself. The idea is to take two randomly augmented views of the same image, a small crop and rotation, say, and train the encoder so that these two views' embeddings are pulled close together (a positive pair), while embeddings of a different image are pushed apart (a negative pair), using only this relative structure, same image versus different image, as the training signal.

Suppose an encoder produces embedding $a = (0.6, 0.8)$ for one crop of a satellite patch, $p = (0.55, 0.83)$ for a second, differently augmented crop of the *same* patch, and $n = (0.1, 0.9)$ for a crop of a genuinely different patch. Using Chapter 16's cosine similarity (Section 16.3.3), $\cos(a, p) = 0.9983$ and $\cos(a, n) = 0.8614$, the same view of the same underlying scene is, as intended, geometrically closer than a different scene.

A contrastive loss converts these similarities into a probability that the positive pair is correctly identified among the candidates, $\Pr(\text{positive}) = e^{\cos(a,p)/\tau} / \left[e^{\cos(a,p)/\tau} + e^{\cos(a,n)/\tau}\right]$, a softmax over similarities exactly analogous to Section 1.4.2's cross-attention softmax, with a temperature parameter τ controlling how sharply the model is pushed to separate positive from negative. With $\tau = 0.1$, $\Pr(\text{positive}) = e^{9.983}/(e^{9.983}+e^{8.614}) = 0.797$, already favoring the true positive pair well above chance (50%) even before any training, and training drives this probability further toward 1 by adjusting the encoder's weights so that same-image crops end up even closer together, and different-image crops even farther apart, than they start.

Once trained this way on a large, unlabeled archive of satellite imagery, the resulting encoder is the frozen backbone Section 1.4.2's transfer-learning example fine-tuned with only a handful of labeled examples, closing the loop between this chapter's two ways of coping with scarce alternative-data labels.

1.5 Case Vignette: Satellite Imagery and the Rise of Alternative Data

The parking-lot car-count example this chapter has used throughout is not hypothetical in its origins. In 2009, brothers Tom and Alex Diamond conceived the idea of counting cars in retailer parking lots from satellite imagery to anticipate revenue, and founded RS Metrics to sell the resulting data, beginning with car counts at McDonald's locations and later covering Lowe's, Home Depot, and dozens of other retailers. The approach reached wider attention in 2010, when UBS analyst Neil Currie purchased parking-lot counts for 100 Walmart store locations and published them ahead of Walmart's earnings release, an early, public demonstration that satellite-derived alternative data could anticipate a reported number before the company itself disclosed it. A later academic study using RS Metrics imagery from 2011 to 2017, covering 44 major U.S. retailers including Walmart, Target, Costco, and Whole Foods, confirmed that year-over-year change in parking-lot car counts reliably predicts quarterly sales, and quantified the resulting informational edge at 4% to 5% in the three trading days surrounding a quarterly earnings announcement, a substantial return for such a short window, corroborating this chapter's own worked example (Section 1.2.3) at genuinely large scale.

A separate study of the same satellite coverage's introduction across retailers found that its capital-market effects went beyond simply rewarding the funds that subscribed to it: retailers who gained satellite coverage subsequently saw more informed short-selling activity, less-informed individual investor buying, and lower stock liquidity around their earnings reports, evidence that unequal access to alternative data can widen the information gap between sophisticated and ordinary investors without necessarily making prices more accurate any faster, precisely the access-asymmetry concern Section 1.3.4 raises about licensing cost and exclusivity.

Satellite-derived alternative data extends well beyond retail. Orbital Insight built a comparable business around commodity markets: by measuring the shadows cast by the floating roofs used on many crude-oil storage tanks, a floating roof sits directly atop the stored oil and its height reveals how full the tank is, the firm estimates inventory levels across more than 27,000 tanks worldwide, work that, during the demand shock of the early 2020s pandemic period, revealed global storage utilization running at roughly 55% of an

estimated six-billion-barrel total capacity, a supply-side data point no single government agency could have produced as quickly. Today, satellite-based financial data services are estimated to generate over $500 million in annual revenue, within a broader alternative-data market Wall Street is estimated to spend roughly $3 billion on annually, and a single institutional satellite subscription can run from $50,000 to $500,000 per year, underscoring Section 1.3.4's point that this edge is available primarily to well-capitalized participants able to pay for exclusive or near-exclusive access.

1.6 Challenges in Alternative Data and Deep Learning in Finance

Several challenges recur across this chapter's techniques and connect directly to themes from earlier chapters.

A short history limits statistical confidence. Section 1.2.3 and Section 1.2.4 fit regressions on only eight quarters, two years of data, because that is genuinely all the history a newly available alternative-data feed typically has; Chapter 6's warnings about overfitting with few observations relative to model complexity apply with particular force here, since a vendor's dataset is often too new to have lived through a full economic cycle.

Non-stationarity compounds with vendor changes. Chapter 6 noted that financial relationships can shift as markets evolve; alternative data adds a second source of instability on top of that, a vendor can silently change its imagery source, its computer-vision model, or its store coverage, so a model's apparent degradation may reflect a data-generating-process change at the vendor rather than a change in the underlying financial relationship.

Deep architectures trade interpretability for representational power. A CNN's learned filters and an RNN's hidden state are far less directly interpretable than the single regression coefficient of Section 1.2.3, a tension Chapter 18 develops in depth, and one that matters more, not less, once a model's input is something as opaque to manual review as a stack of satellite pixels.

Integration is usually the hard part, not modeling. As Section 1.3.4 emphasized, aligning several data sources in time and by firm identity, and confirming each was genuinely available before, not after, the date being modeled, typically consumes more effort in a real alternative-data project than fitting the model itself, the reverse of the balance in the market-data-only settings of Chapters 12 and 13.

Cost and exclusivity concentrate the advantage. Section 1.5's subscription costs mean alternative data's edge accrues disproportionately to well-resourced funds able to license multiple exclusive feeds, an access asymmetry with no real counterpart in the publicly available market and fundamental data most of this book has used.

Model risk governance now has to reach the vendor, not just the model. Chapter 15's three-lines-of-defense framework was built to validate a single model's inputs, outputs, and ongoing performance; an alternative-data pipeline like Section 1.2.4's adds an entire additional layer that framework must now cover, the vendor relationship itself, since a governed model's validation is only as good as the vendor's coverage, delivery timing, and processing methodology sitting upstream of it, none of which the fund directly controls or, in most cases, can fully observe.

1.7 Key Takeaways

Alternative data, satellite imagery, web traffic, card-transaction panels, and the like, fills the row Chapter 1's data-ecosystem table reserved for it: non-traditional sources that proxy for economic activity ahead of its official reporting. Section 1.2.3 showed that even a single such proxy, satellite-derived parking-lot car counts, can explain a large share (87% in the chapter's worked example) of a retailer's quarterly revenue variation, and Section 1.2.4 showed that combining several sources into one regression produces a genuinely useful, if imperfect (an 80% hit rate, not 100%), earnings-surprise signal in the same trading-signal-evaluation framework Chapter 13 established. Before any of this modeling happens, Section 1.3.2 showed that raw alternative-data readings typically need seasonal adjustment and peer-relative normalization to isolate firm-specific signal from calendar effects and sector-wide movements.

Extracting such signals from raw alternative data, rather than from a vendor's already-processed number, requires neural network architectures beyond Chapter 6's plain feedforward network: convolutional neural networks (Section 1.3.3) exploit an image's grid structure through weight-shared filters, recurrent neural

networks exploit a sequence's ordering through a hidden state carried from step to step, and transformer-based self-attention, introduced for text in Chapter 16, increasingly unifies both roles. None of this changes the fact, developed in Section 1.3.4 and the chapter's closing challenges, that alternative data carries its own distinctive quality problems, coverage bias, vendor black-box processing, look-ahead bias from delivery lag, and cost-driven alpha decay, that a modeler must treat as seriously as the modeling itself, a lesson this chapter's case vignette on satellite-derived retail and commodity data illustrated at real-world scale.

Section 1.3.1 showed concretely that respecting real-world data-delivery lag changes which trades a model actually recommends, not merely how good its backtest looks, and Section 1.3.3's autoencoder showed that the same deep learning toolkit that extracts a signal can also be turned, unsupervised, on the data itself to flag a quarter worth double-checking before it is trusted. Section 1.4.1 applied this same lesson to a specific and increasingly important alternative-data category: rather than buying a vendor's already-computed ESG score, whose disagreement with competing providers Chapter 12 documented at 38% to 71% pairwise correlation, a fund can build a narrow, auditable environmental-tone signal directly from a company's own disclosures using Chapter 16's Naive Bayes classifier and Loughran-McDonald tone score, at the cost of adding one more, differently-scoped measurement to a field that already has too many mutually inconsistent ones rather than resolving the disagreement outright.

Section 1.4.2 went one level deeper still: transfer learning lets a CNN's frozen, already-learned features be fine-tuned on a small alternative-data sample rather than requiring millions of labeled images, contrastive pretraining shows how that frozen backbone can itself be trained with no labels at all, graph neural networks let risk propagate across a network of related entities the way Chapter 15's static layering chain could only describe, not compute, and cross-attention, the same mechanism Chapter 16 used to fuse word tokens, fuses image, text, and tabular alternative-data modalities into one representation without an analyst having to hand-specify how much each deserves.

1.8 Suggested Exercises

1. Using Section 1.2.3's method, refit the regression after adding a ninth quarter with car-count growth of -2.0% and revenue growth of -1.5%, and report the new slope, intercept, and R^2.

2. Explain, using Table 1.2, why an R^2 of 0.870 from a single alternative-data proxy is considered strong in this context, contrasting it with Chapter 6's discussion of low signal-to-noise ratios in financial prediction generally.

3. Using Section 1.2.4's fitted coefficients, compute the predicted surprise for a tenth quarter with car, web, and card growth of -2.0%, -1.0%, and -1.5% respectively, and state whether the resulting signal is long or flat.

4. Explain, using Q5 in Table 1.3, why a strongly positive fitted surprise (2.46%) can still be followed by a negative next-day return (-0.4%), and describe one piece of earnings-day information the alternative-data model in this chapter cannot capture.

5. Using Section 1.3.3's convolution example, recompute the 3×3 feature map using a 2×2 filter of all 0.5s instead of all 0.25s, and explain how the result relates to the original feature map.

6. Using the same convolution example, apply 2×2 max pooling (taking the maximum of each non-overlapping 2×2 block) to the original 3×3 feature map's top-left 2×2 block, and state the resulting pooled value.

7. Using Section 1.3.3's RNN example, recompute h_1, h_2, and h_3 if w_h is increased from 0.4 to 0.8 (holding w_x, b, and the inputs fixed), and explain in words how a larger w_h changes how much the hidden state is influenced by earlier time steps versus the current input.

8. Explain, in a short paragraph, why a convolutional neural network's weight sharing makes it far more practical than a plain feedforward network for a 1000×1000-pixel satellite image, referencing the parameter-count argument in Section 1.3.3.

9. Using this chapter's case vignette, explain why the 2010 UBS Walmart parking-lot report is a natural precedent for Section 1.2.3's worked example, and describe, in your own words, why the same informational edge tends to shrink as more funds license the same data source.

10. Pick two of the data-quality issues in Section 1.3.4 and describe a concrete scenario, not already used in the text, in which that issue could cause an alternative-data-based trading or credit model to fail despite technically correct code.

11. Compare this chapter's satellite car-count regression (Section 1.2.3) to Chapter 5's binomial and Black-Scholes-Merton option-pricing models in terms of what each treats as the source of uncertainty, and explain why alternative-data models are typically evaluated by out-of-sample predictive accuracy rather than by no-arbitrage argument.

12. Using Section 1.3.1's method, explain in your own words why the realistic, lagged-card model's higher R^2 (0.873 versus 0.837) does not mean it is simply the better trading model, referencing the difference in which quarters each version actually signals.

13. Using Section 1.3.2's method, recompute the peer-adjusted regression's slope and R^2 if the sector-average car-count growth for Q5 is revised from 2.5% to 5.0% (a sector-wide surge that quarter), holding all other quarters fixed, and explain why the peer-adjusted model's R^2 falls even though Cascade's own reported revenue growth has not changed.

14. A vendor offers a trial dataset covering only the three quarters (of Table 1.3) in which its data predicted the following quarter's earnings surprise correctly. Using Section 1.2.2's discussion, explain what is wrong with evaluating the vendor's data quality using only this trial.

15. Using Section 1.2.2's cost-benefit method, recompute the expected net annual benefit if the fund can only deploy a \$20 million position per signaled event (instead of \$50 million), holding the signal frequency, edge, and data costs fixed, and state whether the subscription is still worth its \$400,000 annual cost.

16. Using Section 1.3.3's autoencoder example, compute the reconstruction error and relative error for Q2 (car $= -1.5$, web $= -0.5$, card $= -1.0$) using the same encoder/decoder weights $w = (0.5, 0.3, 0.2)$, and state whether Q2 would be flagged ahead of Q7 (relative error 39.5%) for data-quality review.

17. Using Section 1.4.2's transfer-learning example, perform a second gradient-descent step starting from the updated weights $w = (0.549, 0.464)$, $b = 0.113$, and report the new prediction, confirming that it continues to move toward the true label of 1.

18. Using Section 1.4.2's graph neural network example, compute h_A and h_D's updated scores using the same weights, and explain, using the network's chain structure $A \to B \to C \to D$, why h_D's score barely moves from its raw feature while h_B's moves substantially.

19. Using Section 1.4.2's cross-attention example, recompute the fused output if the text modality's key becomes $K_{\text{txt}} = (0.8, 0.2)$ (a much less distinctive key vector, more similar to K_{img}), and explain in words why the two attention weights move closer to 50/50.

20. Explain, in a short paragraph, why a graph neural network's mean-aggregation step (Section 1.4.2) is a more direct computational analogue of "guilt by association" than Chapter 15's original, per-account fraud-scoring logistic regression.

21. Using Section 1.4.2's contrastive-pretraining example, recompute Pr(positive) if the temperature τ is raised from 0.1 to 1.0 (holding the two cosine similarities fixed), and explain in words why a higher temperature makes the model's implied preference for the true positive pair less extreme.

22. Using Section 1.3.3's LSTM worked example, recompute the forget gate f_1 and the resulting fraction of c_1 surviving to c_3 (that is, $f_2 \times f_3$) if the forget-gate bias b_f were 0.5 instead of 2.0, holding every other weight fixed, and explain why a smaller bias makes the LSTM behave more like the plain RNN example earlier in this section.

23. Explain, in your own words, why the plain RNN's memory-decay problem is a consequence of its hidden state being fully overwritten by a multiplicative recursion at every step, while the LSTM's cell state update is additive, and describe what would happen to the LSTM's own memory retention if its forget gate were trained to a value close to 0 instead of close to 1.

24. Using Section 1.4.1's method, recompute the tone score for a fifth sentence, "The board remains confident that emissions will decline further even as regulatory headwinds intensify," using the same positive and negative word lists, and explain why this sentence's mixed vocabulary makes its tone score a poor summary of whether the sentence is good or bad news.

25. Explain, in your own words, why Section 1.4.1's in-house tone score and Chapter 12's vendor-style ESG scores are not directly comparable even though both are commonly described as "an ESG measurement," and describe which one of Berg, Kölbel, and Rigobon's three sources of rating divergence, scope, measurement, or weight, this incomparability illustrates.

26. **Challenge (real data).** Download at least two years of weekly Google Trends search-interest data (via `pytrends` or manually from trends.google.com, no API key required) for a public retailer's brand name, and quarterly reported revenue for the same company from the SEC EDGAR XBRL `companyfacts` API (as in Chapter 4's challenge exercise). (a) Following Section 1.2.3's method, regress the company's quarter-over-quarter revenue growth on the average quarter-over-quarter growth in search interest, and report the fitted slope, intercept, and R^2. (b) Using Section 1.3.1's distinction between a realistic, lagged signal and a look-ahead-biased one, identify what reporting lag your search-interest data is actually available at relative to the revenue figure it is meant to predict, and explain whether your regression in (a) would still be usable in real time. (c) Following Section 1.2.2's cost-benefit framework, discuss one alternative-data quality issue from Section 1.3.4 other than look-ahead bias that is specific to Google Trends data, such as index normalization or query ambiguity, and explain how it could produce a spuriously strong or weak regression fit.

1.9 Further Reading

- Berkeley Haas School of Business, "How Hedge Funds Use Satellite Images to Beat Wall Street, and Main Street." A research summary of the RS Metrics parking-lot car-count studies underlying Section 1.2.3 and this chapter's case vignette.

- Katona, Painter, Patatoukas, and Zeng, "On the Capital Market Consequences of Big Data: Evidence from Outer Space," *Journal of Financial and Quantitative Analysis*. The academic study of RS Metrics satellite imagery (2011-2017, 44 U.S. retailers) quantifying the earnings-window informational edge discussed in Section 1.5.

- Feng and Fay, "An Empirical Investigation of Forward-Looking Retailer Performance Using Parking Lot Traffic Data Derived from Satellite Imagery," *Journal of Retailing*. A complementary study of satellite-derived parking-lot data's power to predict retailer sales, underlying Section 1.2.3's worked example.

- LeCun, Bengio, and Hinton, "Deep Learning," *Nature*. A survey covering convolutional and recurrent neural network architectures underlying Section 1.3.3.

- Goodfellow, Bengio, and Courville, *Deep Learning*. A comprehensive textbook treatment of convolution, recurrence, and the attention mechanisms Chapter 16 and Section 1.3.3 both draw on.

- Hochreiter and Schmidhuber, "Long Short-Term Memory," *Neural Computation*. The original paper introducing the LSTM architecture mentioned as an extension of Section 1.3.3's plain recurrent network.

- Dosovitskiy et al., "An Image Is Worth 16x16 Words: Transformers for Image Recognition at Scale," *International Conference on Learning Representations*. The original vision transformer paper underlying Section 1.3.3's discussion of transformers as a unifying architecture.

- Kolanovic and Krishnamachari, "Big Data and AI Strategies," J.P. Morgan. An industry survey of alternative data sources and their use in systematic investing, covering the categories in Table 1.1.
- Berg, Kölbel, and Rigobon, "Aggregate Confusion: The Divergence of ESG Ratings," *Review of Finance*. The empirical study of cross-provider ESG rating disagreement discussed in Chapter 12 and revisited in Section 1.4.1 from the standpoint of building an in-house text-derived signal rather than buying a vendor's score.
- Katona, Painter, Patatoukas, and Zeng, "On the Capital Market Consequences of Big Data: Evidence from Outer Space" (same source as above). Also documents the short-selling, liquidity, and information-asymmetry effects of satellite coverage discussed in this chapter's expanded case vignette.
- Hinton and Salakhutdinov, "Reducing the Dimensionality of Data with Neural Networks," *Science*. A foundational paper on autoencoders, underlying Section 1.3.3's reconstruction-error anomaly-detection example.
- Pan and Yang, "A Survey on Transfer Learning," *IEEE Transactions on Knowledge and Data Engineering*. A comprehensive survey of the pretrained-backbone fine-tuning approach worked through in Section 1.4.2.
- Kipf and Welling, "Semi-Supervised Classification with Graph Convolutional Networks," *International Conference on Learning Representations*. A foundational paper on the graph neural network mean-aggregation update used in Section 1.4.2's shell-company network example.
- Chen, Kornblith, Norouzi, and Hinton, "A Simple Framework for Contrastive Learning of Visual Representations," *International Conference on Machine Learning*. The SimCLR paper underlying Section 1.4.2's contrastive-pretraining worked example.
- He, Zhang, Ren, and Sun, "Deep Residual Learning for Image Recognition," *IEEE Conference on Computer Vision and Pattern Recognition*. The original ResNet paper underlying Section 1.3.3's discussion of shortcut connections and the vanishing-gradient problem in deep CNNs.